

SOUTENANCE — TITRE RNCP

Détection de fraude bancaire par transaction carte

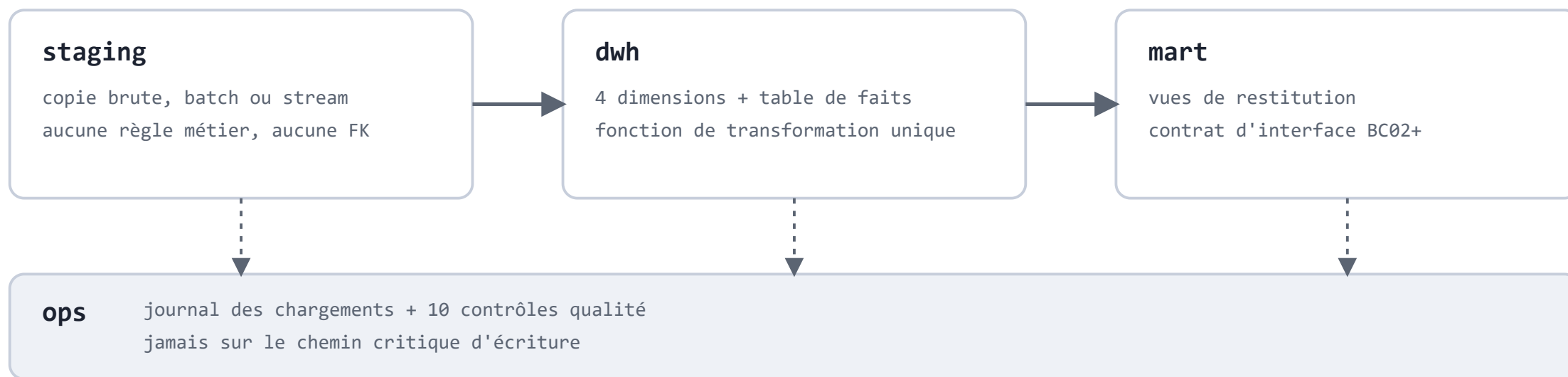
Pipeline Machine Learning & Deep Learning de bout en bout — du socle de données à l'API de production

Blocs **BC01** à **BC05** · 5 000 clients simulés · 400 000 transactions · github.com/k-b-mansour/Detection-Fraude-

Un entrepôt en étoile, une seule porte d'entrée

- **Données synthétiques**, pas un CSV Kaggle : contrôle total du schéma et des typologies de fraude
- **4 couches** — staging → dwh → mart → ops — batch et streaming ne peuvent pas diverger
- **Une seule fonction SQL** de transformation, appelée par le script batch *et* le consumer Kafka

Architecture



Résultats vérifiés

400 000

transactions ingérées (batch)

10 / 10

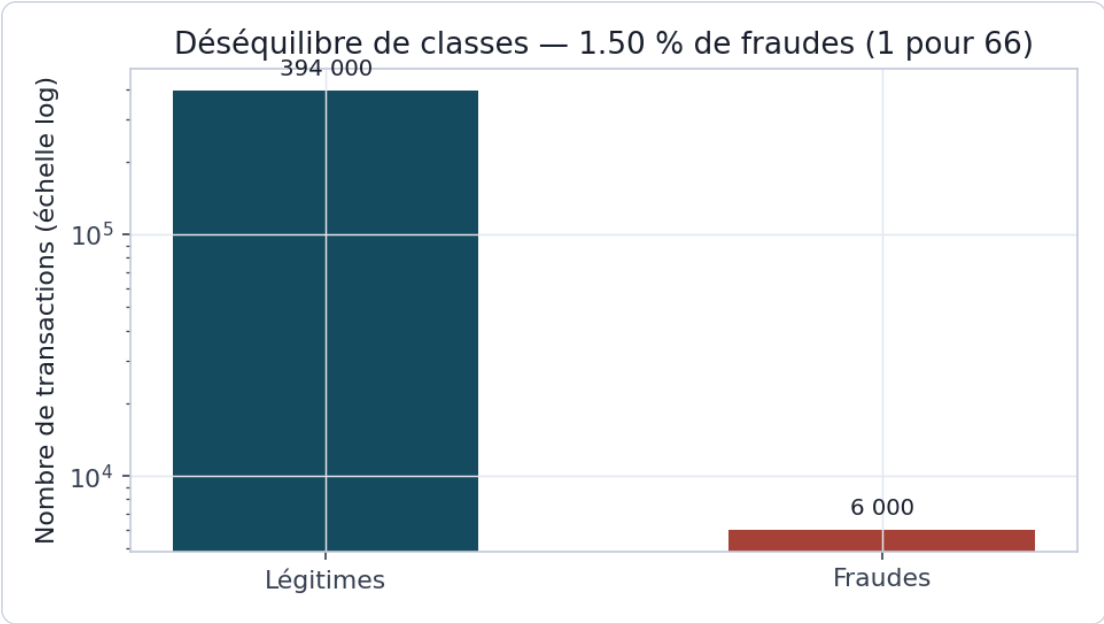
contrôles qualité au vert

3

incidents réels corrigés (largeur de clé, dédup
Redis, membre Inconnu)

Garanties : idempotence (batch et stream), DLQ pour les messages invalides, membre « Inconnu » plutôt qu'une perte silencieuse.

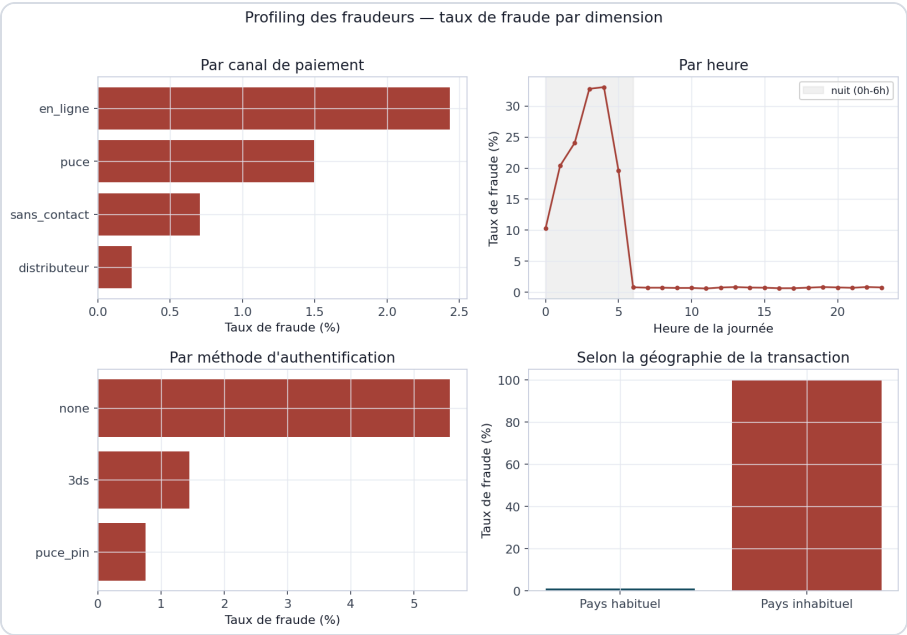
Un déséquilibre extrême



1,50 %
de transactions frauduleuses

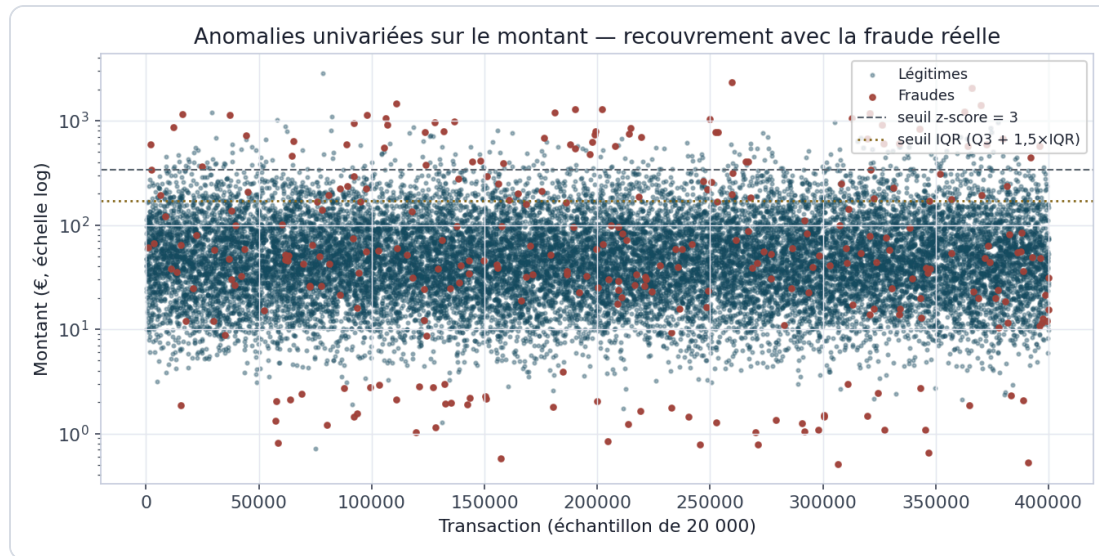
1 : 66
déséquilibre de classes

Profiling des fraudeurs



33 % de fraude à 4h du matin — contre moins de 1 % en journée.

La limite d'une seule variable



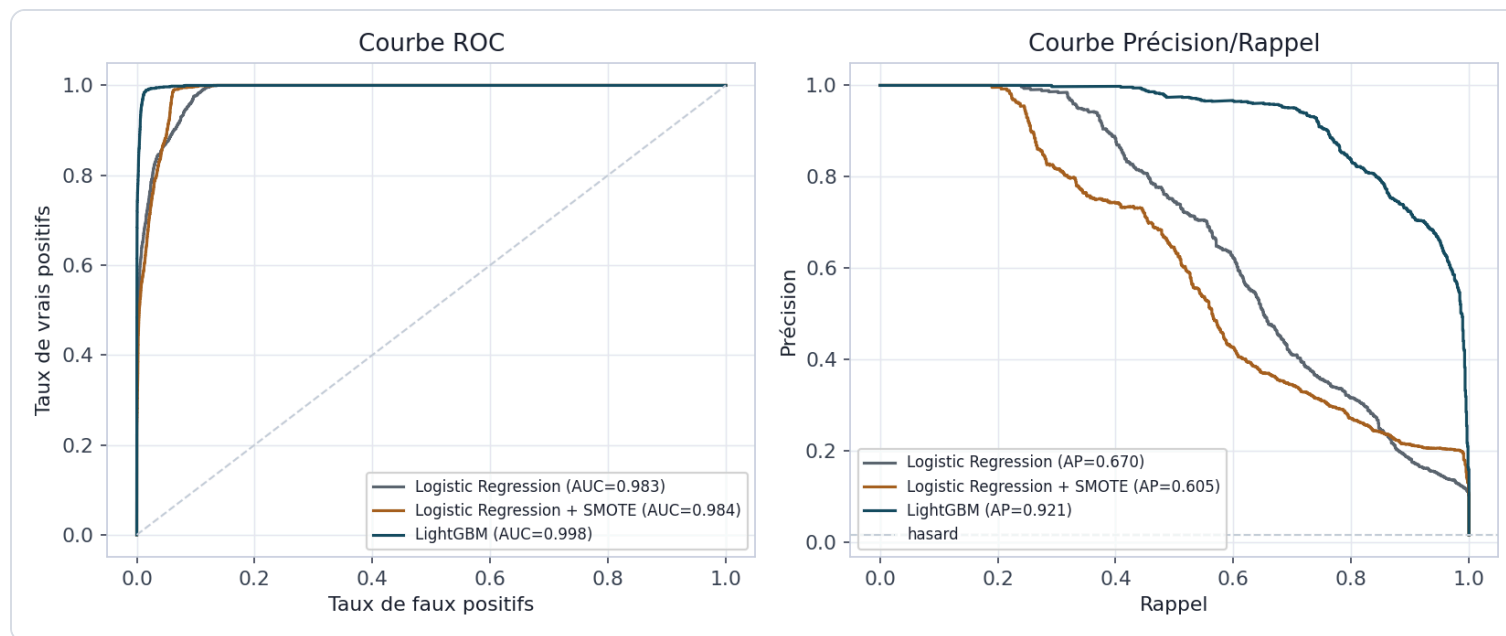
- Détecteur z-score : **21 % de rappel** seulement
- La fraude porte **6 signatures différentes** — aucune variable isolée ne les capture toutes
- → justifie le passage aux **modèles multivariés** (BC03)

Trois configurations, un protocole commun

CONFIGURATION	DÉSÉQUILIBRE	RÉSULTAT NOTABLE
Logistic Regression	Aucun	F1 = 0,580
Logistic Regression + SMOTE	SMOTE 30 %	F1 = 0,368 (dégradé)
LightGBM	scale_pos_weight	AUC = 0,998

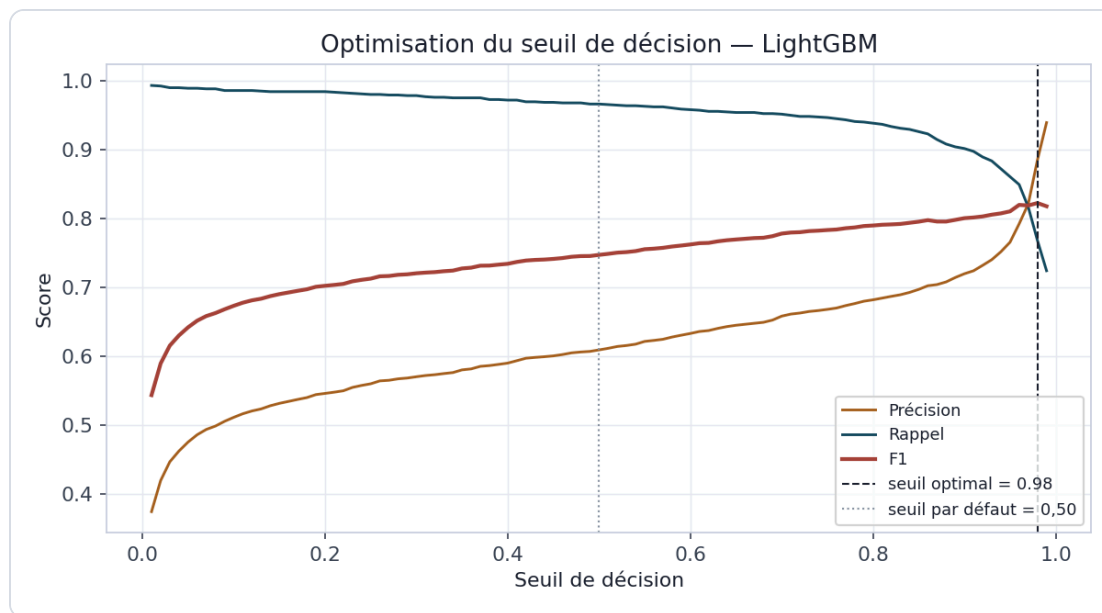
Split **temporel** (pas aléatoire) : le modèle ne voit jamais le futur pendant l'entraînement.

LightGBM se détache nettement



Sous un tel déséquilibre, l'**average precision** (0,921 vs 0,670) départage les modèles — pas l'AUC-ROC, presque saturée pour les trois.

Seuil optimisé & explicabilité SHAP

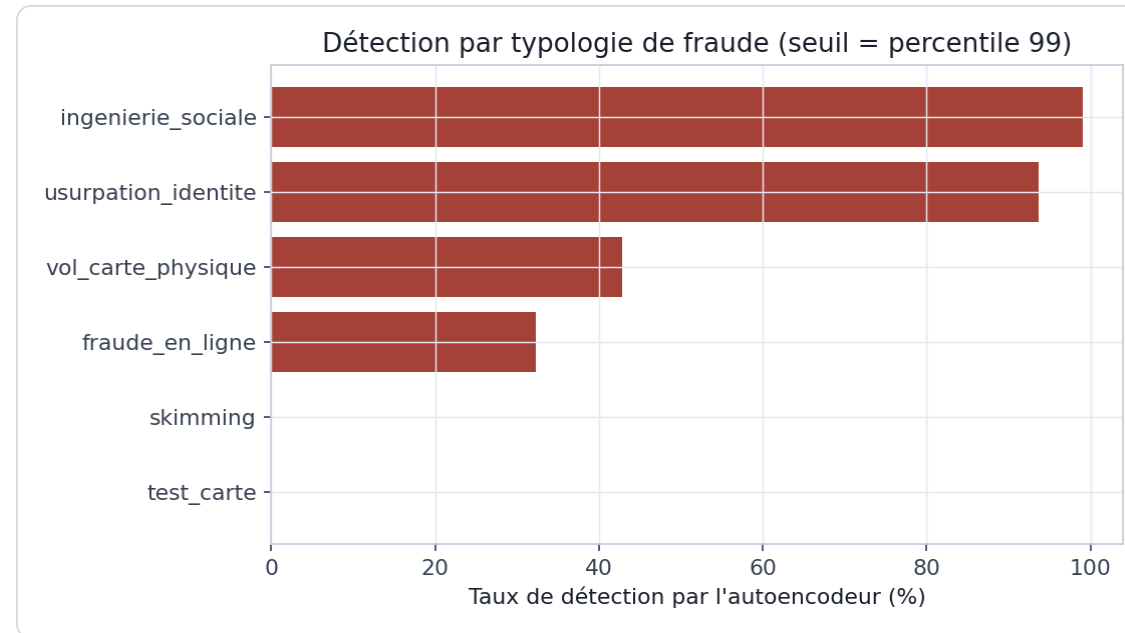


→ Seuil par défaut (0,50) : F1 = 0,747

→ Seuil optimisé (**0,98**) : F1 = **0,822**

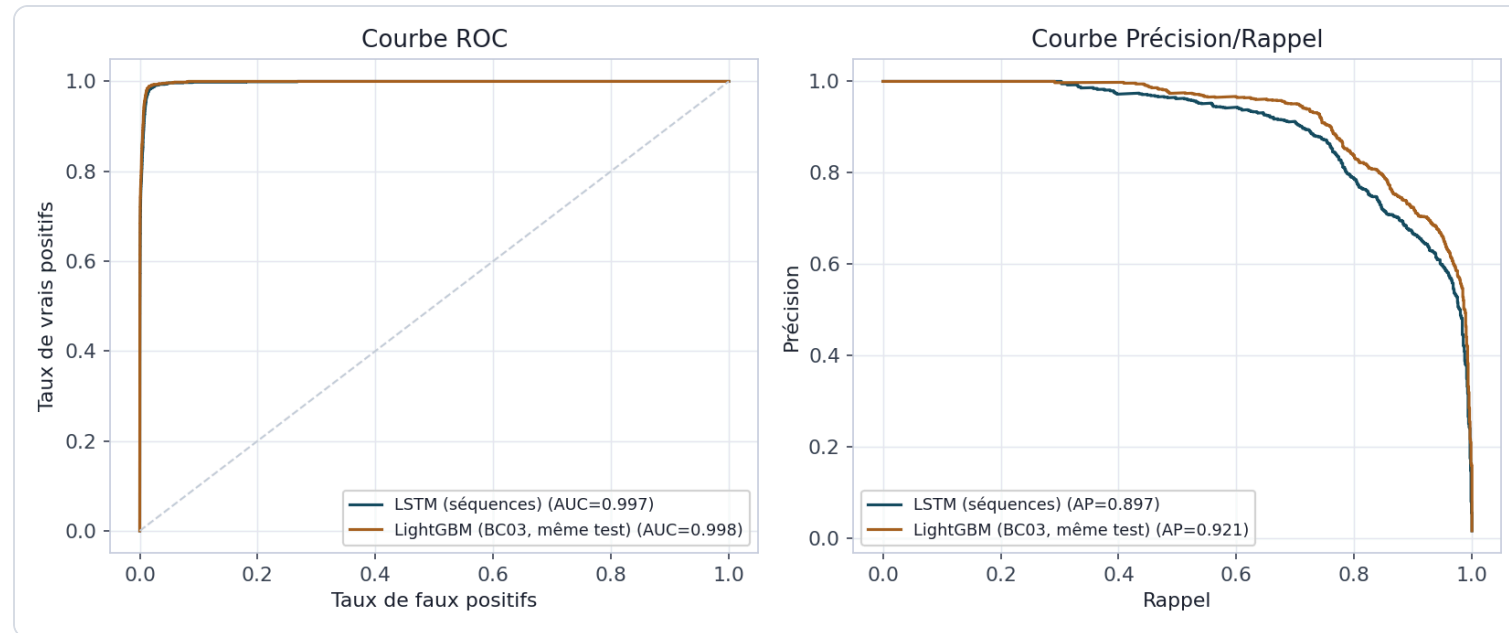
→ SHAP par décision → conformité **RGPD Art. 22**

Autoencodeur : anomalies ponctuelles vs contextuelles



Détecte 99 % des fraudes **extrêmes** (montant aberrant) sans jamais voir le label — mais **0 %** des fraudes discrètes (skimming, test carte).

Réseau de séquences (LSTM) vs LightGBM



LSTM : AUC **0,997**, quasi à parité avec LightGBM (0,998) — sur le *même* sous-ensemble de test.

Pourquoi pas de gain temporel ici

- Chaque typologie de fraude (BC01) est définie par **une seule transaction**, jamais une séquence corrélée
- skimming — pensé comme une rafale — est en réalité généré de façon indépendante
- → limite du **générateur**, pas de l'architecture séquentielle

Scoring temps réel, expliqué

24-60 ms

latence observée (conteneur)

< 200 ms

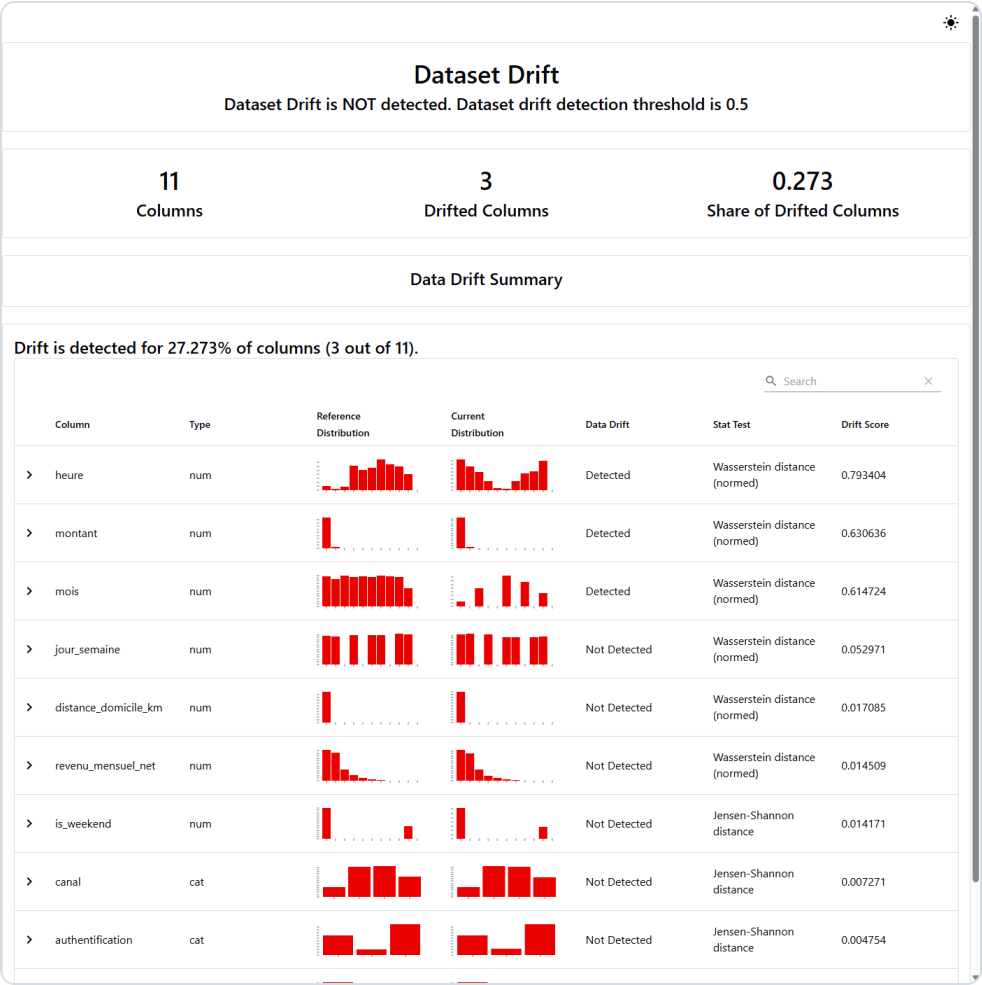
contrainte réglementaire

7 / 7

tests automatisés

SHAP recalculé **à la volée** à chaque requête — score, niveau de risque, top 5 facteurs.

Monitoring de dérive (Evidently)



- Scénario normal : **9,1 %** de dérive — pas d'alerte
- Scénario simulé : **27,3 %** — alerte déclenchée
- Seuil d'alerte : 20 %

CI/CD — un pipeline réellement exécuté

RUN	RÉSULTAT	CAUSE
#1	Échec	pytest nu (sys.path)
#2-3	Échec	Nom d'image GHCR invalide
#4	Succès	Test + build + publication GHCR

Image publiée : `ghcr.io/k-b-mansour/fraud-api` — sur de vrais runners GitHub Actions.